JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ZipInputStream

java.lang.Object
    java.io.InputStream
        java.io.FilterInputStream
            java.util.zip.InflaterInputStream
                java.util.zip.ZipInputStream

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

Direct Known Subclasses:

`JarInputStream`

```
public class ZipInputStream
extends InflaterInputStream
```

An input stream for reading compressed and uncompressed ZIP file entries from a stream of bytes in the ZIP file format.

Unless otherwise noted, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

Reading Zip File Entries

The `getNextEntry()` method is used to read the next ZIP file entry (Local file (LOC) header record in the ZIP format) and position the stream at the entry's file data. The file data may read using one of the `ZipInputStream` read methods such as `read` or `readAllBytes()`. For example:

```
Path jar = Path.of("foo.jar");
try (InputStream is = Files.newInputStream(jar);
     ZipInputStream zis = new ZipInputStream(is)) {
    ZipEntry ze;
    while ((ze = zis.getNextEntry()) != null) {
       var bytes = zis.readAllBytes();
       System.out.printf("Entry: %s, bytes read: %s%n", ze.getName(),
               bytes.length);
    }
}
```

API Note:

The LOC header contains metadata about the ZIP file entry. `ZipInputStream` does not read the Central directory (CEN) header for the entry and therefore will not have access to its metadata such as the external file attributes.

ZipFile may be used when the information stored within the CEN header is required.

Since:

1.1

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	**`CENATT`**	Central directory (CEN) header internal file attributes field offset.
`static final int`	**`CENATX`**	Central directory (CEN) header external file attributes field offset.
`static final int`	**`CENCOM`**	Central directory (CEN) header comment length field offset.
`static final int`	**`CENCRC`**	Central directory (CEN) header uncompressed file crc-32 value field offset.
`static final int`	**`CENDSK`**	Central directory (CEN) header disk number start field offset.
`static final int`	**`CENEXT`**	Central directory (CEN) header extra field length field offset.
`static final int`	**`CENFLG`**	Central directory (CEN) header encrypt, decrypt flags field offset.
`static final int`	**`CENHDR`**	Central directory (CEN) header size in bytes (including signature).
`static final int`	**`CENHOW`**	Central directory (CEN) header compression method field offset.

static final int	**CENLEN**	Central directory (CEN) header uncompressed size field offset.
static final int	**CENNAM**	Central directory (CEN) header filename length field offset.
static final int	**CENOFF**	Central directory (CEN) header LOC header offset field offset.
static final long	**CENSIG**	Central directory (CEN) header signature.
static final int	**CENSIZ**	Central directory (CEN) header compressed size field offset.
static final int	**CENTIM**	Central directory (CEN) header modification time field offset.
static final int	**CENVEM**	Central directory (CEN) header version made by field offset.
static final int	**CENVER**	Central directory (CEN) header version needed to extract field offset.
static final int	**ENDCOM**	End of central directory (END) header ZIP file comment length field offset.
static final int	**ENDHDR**	End of central directory (END) header size in bytes (including signature).
static final int	**ENDOFF**	End of central directory (END) header offset for the first CEN header field offset.
static final long	**ENDSIG**	

		End of central directory (END) header signature.
`static final int`	**`ENDSIZ`**	End of central directory (END) header central directory size in bytes field offset.
`static final int`	**`ENDSUB`**	End of central directory (END) header number of entries on this disk field offset.
`static final int`	**`ENDTOT`**	End of central directory (END) header total number of entries field offset.
`static final int`	**`EXTCRC`**	Extra local (EXT) header uncompressed file crc-32 value field offset.
`static final int`	**`EXTHDR`**	Extra local (EXT) header size in bytes (including signature).
`static final int`	**`EXTLEN`**	Extra local (EXT) header uncompressed size field offset.
`static final long`	**`EXTSIG`**	Extra local (EXT) header signature.
`static final int`	**`EXTSIZ`**	Extra local (EXT) header compressed size field offset.
`static final int`	**`LOCCRC`**	Local file (LOC) header uncompressed file crc-32 value field offset.
`static final int`	**`LOCEXT`**	Local file (LOC) header extra field length field offset.
`static final int`	**`LOCFLG`**	

		Local file (LOC) header general purpose bit flag field offset.
static final int	**LOCHDR**	Local file (LOC) header size in bytes (including signature).
static final int	**LOCHOW**	Local file (LOC) header compression method field offset.
static final int	**LOCLEN**	Local file (LOC) header uncompressed size field offset.
static final int	**LOCNAM**	Local file (LOC) header filename length field offset.
static final long	**LOCSIG**	Local file (LOC) header signature.
static final int	**LOCSIZ**	Local file (LOC) header compressed size field offset.
static final int	**LOCTIM**	Local file (LOC) header modification time field offset.
static final int	**LOCVER**	Local file (LOC) header version needed to extract field offset.

Fields declared in class InflaterInputStream

`buf`, `inf`, `len`

Fields declared in class FilterInputStream

`in`

Constructor Summary

Constructors

Constructor	Description
`ZipInputStream(InputStream in)`	Creates a new ZIP input stream.
`ZipInputStream(InputStream in, Charset charset)`	Creates a new ZIP input stream.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`int`	`available()`	Returns 0 when end of stream is detected for the current ZIP entry or `closeEntry()` has been called on the current ZIP entry, otherwise returns 1.
`void`	`close()`	Closes this input stream and releases any system resources associated with the stream.
`void`	`closeEntry()`	Closes the current ZIP entry and positions the stream for reading the next entry.
`protected ZipEntry`	`createZipEntry(String name)`	Creates a new `ZipEntry` object for the specified entry name.
`ZipEntry`	`getNextEntry()`	Reads the next ZIP file entry and positions the stream at the beginning of the entry data.
`int`	`read()`	Reads the next byte of data from the input

		stream for the current ZIP entry.
int	**read**(byte[] b, int off, int len)	Reads the requested number of bytes from the input stream into the given byte array for the current ZIP entry returning the number of inflated bytes.
byte[]	**readAllBytes**()	Reads all remaining bytes from the input stream for the current ZIP entry.
int	**readNBytes**(byte[] b, int off, int len)	Reads the requested number of bytes from the input stream into the given byte array for the current ZIP entry returning the number of inflated bytes.
byte[]	**readNBytes**(int len)	Reads up to a specified number of bytes from the input stream for the current ZIP entry.
long	**skip**(long n)	Skips over and discards n bytes of data from this input stream for the current ZIP entry.
void	**skipNBytes**(long n)	Skips over and discards exactly n bytes of data from this input stream for the current ZIP entry.
long	**transferTo**(OutputStream out)	Reads all bytes from this input stream for the current ZIP entry and writes the bytes to the given output stream in

	the order that they are read.

Methods declared in class InflaterInputStream

`fill`, `mark`, `markSupported`, `reset`

Methods declared in class FilterInputStream

`read`

Methods declared in class InputStream

`nullInputStream`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Field Details

LOCSIG

```
static final long LOCSIG
```

Local file (LOC) header signature.

See Also:

Constant Field Values

EXTSIG

```
static final long EXTSIG
```

Extra local (EXT) header signature.

See Also:

Constant Field Values

CENSIG

```
static final long CENSIG
```

Central directory (CEN) header signature.

See Also:

Constant Field Values

ENDSIG

```
static final long ENDSIG
```

End of central directory (END) header signature.

See Also:

Constant Field Values

LOCHDR

```
static final int LOCHDR
```

Local file (LOC) header size in bytes (including signature).

See Also:

Constant Field Values

EXTHDR

```
static final int EXTHDR
```

Extra local (EXT) header size in bytes (including signature).

See Also:

Constant Field Values

CENHDR

```
static final int CENHDR
```

Central directory (CEN) header size in bytes (including signature).

See Also:

Constant Field Values

ENDHDR

```
static final int ENDHDR
```

End of central directory (END) header size in bytes (including signature).

See Also:

Constant Field Values

LOCVER

`static final int LOCVER`

Local file (LOC) header version needed to extract field offset.

See Also:

Constant Field Values

LOCFLG

`static final int LOCFLG`

Local file (LOC) header general purpose bit flag field offset.

See Also:

Constant Field Values

LOCHOW

`static final int LOCHOW`

Local file (LOC) header compression method field offset.

See Also:

Constant Field Values

LOCTIM

`static final int LOCTIM`

Local file (LOC) header modification time field offset.

See Also:

Constant Field Values

LOCCRC

`static final int LOCCRC`

Local file (LOC) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

LOCSIZ

```
static final int LOCSIZ
```

Local file (LOC) header compressed size field offset.

See Also:

Constant Field Values

LOCLEN

```
static final int LOCLEN
```

Local file (LOC) header uncompressed size field offset.

See Also:

Constant Field Values

LOCNAM

```
static final int LOCNAM
```

Local file (LOC) header filename length field offset.

See Also:

Constant Field Values

LOCEXT

```
static final int LOCEXT
```

Local file (LOC) header extra field length field offset.

See Also:

Constant Field Values

EXTCRC

```
static final int EXTCRC
```

Extra local (EXT) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

EXTSIZ

```
static final int EXTSIZ
```

Extra local (EXT) header compressed size field offset.

See Also:

Constant Field Values

EXTLEN

`static final int EXTLEN`

Extra local (EXT) header uncompressed size field offset.

See Also:

Constant Field Values

CENVEM

`static final int CENVEM`

Central directory (CEN) header version made by field offset.

See Also:

Constant Field Values

CENVER

`static final int CENVER`

Central directory (CEN) header version needed to extract field offset.

See Also:

Constant Field Values

CENFLG

`static final int CENFLG`

Central directory (CEN) header encrypt, decrypt flags field offset.

See Also:

Constant Field Values

CENHOW

`static final int CENHOW`

Central directory (CEN) header compression method field offset.

See Also:

Constant Field Values

CENTIM

```
static final int CENTIM
```

Central directory (CEN) header modification time field offset.

See Also:

Constant Field Values

CENCRC

```
static final int CENCRC
```

Central directory (CEN) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

CENSIZ

```
static final int CENSIZ
```

Central directory (CEN) header compressed size field offset.

See Also:

Constant Field Values

CENLEN

```
static final int CENLEN
```

Central directory (CEN) header uncompressed size field offset.

See Also:

Constant Field Values

CENNAM

```
static final int CENNAM
```

Central directory (CEN) header filename length field offset.

See Also:

Constant Field Values

CENEXT

```
static final int CENEXT
```

Central directory (CEN) header extra field length field offset.

See Also:

Constant Field Values

CENCOM

`static final int CENCOM`

Central directory (CEN) header comment length field offset.

See Also:

Constant Field Values

CENDSK

`static final int CENDSK`

Central directory (CEN) header disk number start field offset.

See Also:

Constant Field Values

CENATT

`static final int CENATT`

Central directory (CEN) header internal file attributes field offset.

See Also:

Constant Field Values

CENATX

`static final int CENATX`

Central directory (CEN) header external file attributes field offset.

See Also:

Constant Field Values

CENOFF

`static final int CENOFF`

Central directory (CEN) header LOC header offset field offset.

See Also:

Constant Field Values

ENDSUB

```
static final int ENDSUB
```

End of central directory (END) header number of entries on this disk field offset.

See Also:

Constant Field Values

ENDTOT

```
static final int ENDTOT
```

End of central directory (END) header total number of entries field offset.

See Also:

Constant Field Values

ENDSIZ

```
static final int ENDSIZ
```

End of central directory (END) header central directory size in bytes field offset.

See Also:

Constant Field Values

ENDOFF

```
static final int ENDOFF
```

End of central directory (END) header offset for the first CEN header field offset.

See Also:

Constant Field Values

ENDCOM

```
static final int ENDCOM
```

End of central directory (END) header ZIP file comment length field offset.

See Also:

Constant Field Values

Constructor Details

ZipInputStream

```
public ZipInputStream(InputStream in)
```

Creates a new ZIP input stream.

The UTF-8 `charset` is used to decode the entry names.

Parameters:

`in` - the actual input stream

ZipInputStream

```
public ZipInputStream(InputStream in,
                      Charset charset)
```

Creates a new ZIP input stream.

Parameters:

`in` - the actual input stream

`charset` - The charset to be used to decode the ZIP entry name (ignored if the language encoding bit of the ZIP entry's general purpose bit flag is set).

Since:

1.7

Method Details

getNextEntry

```
public ZipEntry getNextEntry()
                      throws IOException
```

Reads the next ZIP file entry and positions the stream at the beginning of the entry data.

Returns:

the next ZIP file entry, or null if there are no more entries

Throws:

`ZipException` - if a ZIP file error has occurred

`IOException` - if an I/O error has occurred

closeEntry

```
public void closeEntry()
                throws IOException
```

Closes the current ZIP entry and positions the stream for reading the next entry.

Throws:

`ZipException` - if a ZIP file error has occurred

`IOException` - if an I/O error has occurred

available

```
public int available()
             throws IOException
```

Returns 0 when end of stream is detected for the current ZIP entry or `closeEntry()` has been called on the current ZIP entry, otherwise returns 1.

Programs should not count on this method to return the actual number of bytes that could be read without blocking.

Overrides:

`available` in class `InflaterInputStream`

Returns:

0 when end of stream is detected for the current ZIP entry or `closeEntry()` has been called on the current ZIP entry, otherwise 1.

Throws:

`IOException` - if an I/O error occurs.

read

```
public int read()
         throws IOException
```

Reads the next byte of data from the input stream for the current ZIP entry. This method will block until enough input is available for decompression.

Overrides:

`read` in class `InflaterInputStream`

Returns:

the byte read, or -1 if the end of the stream is reached

Throws:

`IOException` - if an I/O error has occurred

See Also:

FilterInputStream.in

readAllBytes

```
public byte[] readAllBytes()
                    throws IOException
```

Reads all remaining bytes from the input stream for the current ZIP entry. This method blocks until all remaining bytes have been read and end of stream is detected, or an exception is thrown. This method does not close the input stream.

When this stream reaches end of stream, further invocations of this method will return an empty byte array.

Note that this method is intended for simple cases where it is convenient to read all bytes into a byte array. It is not intended for reading input streams with large amounts of data.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes have been read. Consequently, the input stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Overrides:

readAllBytes in class InputStream

Returns:

a byte array containing the bytes read from this input stream

Throws:

OutOfMemoryError - if an array of the required size cannot be allocated.

IOException - if an I/O error occurs

Since:

9

readNBytes

```
public byte[] readNBytes(int len)
                  throws IOException
```

Reads up to a specified number of bytes from the input stream for the current ZIP entry. This method blocks until the requested number of bytes has been read, end of stream is detected, or an exception is thrown. This method does not close the input stream.

The length of the returned array equals the number of bytes read from the stream. If `len` is zero, then no bytes are read and an empty byte array is returned. Otherwise, up to `len` bytes are read from the stream. Fewer than `len` bytes may be read if end of stream is encountered.

When this stream reaches end of stream, further invocations of this method will return an empty byte array.

Note that this method is intended for simple cases where it is convenient to read the specified number of bytes into a byte array. The total amount of memory allocated by this method is proportional to the number of bytes read from the stream which is bounded by `len`. Therefore, the method may be safely called with very large values of `len` provided sufficient memory is available.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes have been read. Consequently, the input stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Overrides:

`readNBytes` in class `InputStream`

Implementation Note:

This method calls `super.readNBytes(int len)`.

Parameters:

`len` - the maximum number of bytes to read

Returns:

a byte array containing the bytes read from this input stream

Throws:

`OutOfMemoryError` - if an array of the required size cannot be allocated.

`IOException` - if an I/O error occurs

Since:

11

readNBytes

```
public int readNBytes(byte[] b,
                      int off,
                      int len)
               throws IOException
```

Reads the requested number of bytes from the input stream into the given byte array for the current ZIP entry returning the number of inflated bytes. This method blocks until `len` bytes of input data have been read, end of

stream is detected, or an exception is thrown. The number of bytes actually read, possibly zero, is returned. This method does not close the input stream.

In the case where end of stream is reached before `len` bytes have been read, then the actual number of bytes read will be returned. When this stream reaches end of stream, further invocations of this method will return zero.

If `len` is zero, then no bytes are read and `0` is returned; otherwise, there is an attempt to read up to `len` bytes.

The first byte read is stored into element `b[off]`, the next one in to `b[off+1]`, and so on. The number of bytes read is, at most, equal to `len`. Let *k* be the number of bytes actually read; these bytes will be stored in elements `b[off]` through `b[off+`*k*`-1]`, leaving elements `b[off+`*k*` ]` through `b[off+len-1]` unaffected.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes of `b` have been updated with data from the input stream. Consequently, the input stream and `b` may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Overrides:

`readNBytes` in class `InputStream`

Parameters:

`b` - the byte array into which the data is read

`off` - the start offset in `b` at which the data is written

`len` - the maximum number of bytes to read

Returns:

the actual number of bytes read into the buffer

Throws:

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

`IOException` - if an I/O error occurs

Since:

9

skipNBytes

```
public void skipNBytes(long n)
                throws IOException
```

Skips over and discards exactly `n` bytes of data from this input stream for the current ZIP entry. If `n` is zero, then no bytes are skipped. If `n` is negative, then no bytes are skipped. Subclasses may handle the negative value differently.

This method blocks until the requested number of bytes has been skipped, end of file is reached, or an exception is thrown.

If end of stream is reached before the stream is at the desired position, then an `EOFException` is thrown.

If an I/O error occurs, then the input stream may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Overrides:

`skipNBytes` in class `InputStream`

Parameters:

`n` - the number of bytes to be skipped.

Throws:

`IOException` - if the stream cannot be positioned properly or if an I/O error occurs.

Since:

12

See Also:

`InputStream.skip(long)`

transferTo

```
public long transferTo(OutputStream out)
                throws IOException
```

Reads all bytes from this input stream for the current ZIP entry and writes the bytes to the given output stream in the order that they are read. On return, this input stream will be at end of stream. This method does not close either stream.

This method may block indefinitely reading from the input stream, or writing to the output stream. The behavior for the case where the input and/or output stream is *asynchronously closed*, or the thread interrupted during the transfer, is highly input and output stream specific, and therefore not specified.

If the total number of bytes transferred is greater than Long.MAX_VALUE, then `Long.MAX_VALUE` will be returned.

If an I/O error occurs reading from the input stream or writing to the output stream, then it may do so after some bytes have been read or written. Consequently, the input stream may not be at end of stream and one, or both, streams may be in an inconsistent state. It is strongly recommended that both streams be promptly closed if an I/O error occurs.

Overrides:

`transferTo` in class `InputStream`

Parameters:

`out` - the output stream, non-null

Returns:

the number of bytes transferred

Throws:

`IOException` - if an I/O error occurs when reading or writing

Since:

9

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads the requested number of bytes from the input stream into the given byte array for the current ZIP entry returning the number of inflated bytes. If `len` is not zero, the method blocks until some input is available; otherwise, no bytes are read and `0` is returned.

If the current entry is compressed and this method returns a nonzero integer *n* then `buf[off]` through `buf[off+`*n*`-1]` contain the uncompressed data. The content of elements `buf[off+`*n*`]` through `buf[off+`*len*`-1]` is undefined, contrary to the specification of the `InputStream` superclass, so an implementation is free to modify these elements during the inflate operation. If this method returns `-1` or throws an exception then the content of `buf[off]` through `buf[off+`*len* `-1]` is undefined.

Overrides:

`read` in class `InflaterInputStream`

Parameters:

`b` - the buffer into which the data is read

`off` - the start offset in the destination array `b`

`len` - the maximum number of bytes read

Returns:

the actual number of bytes read, or -1 if the end of the entry is reached

Throws:

`IndexOutOfBoundsException` - if `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

`ZipException` - if a ZIP file error has occurred

`IOException` - if an I/O error has occurred

See Also:

`FilterInputStream.in`

skip

```
public long skip(long n)
          throws IOException
```

Skips over and discards `n` bytes of data from this input stream for the current ZIP entry.

Overrides:

`skip` in class `InflaterInputStream`

Parameters:

`n` - the number of bytes to skip

Returns:

the actual number of bytes skipped

Throws:

`ZipException` - if a ZIP file error has occurred

`IOException` - if an I/O error has occurred

`IllegalArgumentException` - if `n < 0`

See Also:

`InputStream.skipNBytes(long)`

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

close in interface Closeable

Overrides:

close in class InflaterInputStream

Throws:

IOException - if an I/O error has occurred

See Also:

FilterInputStream.in

createZipEntry

```
protected ZipEntry createZipEntry(String name)
```

Creates a new ZipEntry object for the specified entry name.

Parameters:

name - the ZIP file entry name

Returns:

the ZipEntry just created

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.